

PROJECT SUMMARY

TalkToJesus — A Bilingual Voice-First AI Spiritual Companion

Manish Rachakonda (2023EBCS668) · BSc Computer Science (Online Mode) · BITS Pilani

Supervisor: Swapnil Saurav · Academic Year 2025–2026 Repository:

<https://github.com/manish-gitx/rn-final-manish>

Prepared so that the viva examiner can understand the project before the session. The full argument, evidence and limitations are in the final project report.

1. The problem

Telugu-language Bible applications have been downloaded by millions of users. They are well made, and they all share one property: content flows one way. A believer can read a passage, follow a plan or play a recorded hymn, but cannot ask what any of it means for the decision in front of them tonight.

Four gaps follow. Communication is one-way. Pastoral conversation requires another person to be available, scheduled and willing to be approached. Guidance is not aware of the asker's situation. And general-purpose AI assistants, which can hold a conversation, are neither grounded in scripture nor idiomatic in Telugu — addressing a believer as నా బిడ్డ is not something a literal translation gets right.

2. Unique Value Proposition

Every existing option covers some of the four. None covers all four at once.

TalkToJesus is conversational, scripture-grounded, regional-language and voice-first simultaneously — and it is the only one of them that can tell you, per turn, where its seconds go.

Set against the alternatives surveyed in Phase 1:

System	Conversational	Scripture-grounded	Telugu	Voice-first
YouVersion Bible App	No	Yes	Partial	No

System	Conversational	Scripture-grounded	Telugu	Voice-first
Telugu Bible applications	No	Yes	Yes	No
Pray.com	No	Yes	No	Partial
General AI assistants	Yes	No	Weak	Partial
Glorify / Abide	No	Yes	No	Partial
TalkToJesus	Yes	Yes	Yes	Yes

The second half of the proposition is the engineering one and matters more for assessment than for a user. Every conversational turn records the duration of transcription, generation and speech synthesis separately, alongside the total. The system therefore measures itself. As Section 5 records, that instrumentation produced a result which contradicted the project’s own earlier documentation — which is the point of building it.

3. The solution

A user opens the application, presses one control, speaks in English or Telugu, and hears a spoken reply written in the first person, addressed as “My child” or “నా బిడ్డ”, citing scripture by book, chapter and verse, and closing with a short prayer.

Around that: a bilingual Bible reader with six translations including two Telugu ones and an offline cache; twelve public-domain hymns bundled to play with no network; a private transcript of previous conversations; and a ₹499/month subscription that lifts a three-conversation free tier.

4. Architecture in brief

Three tiers. A **Flutter** client for iOS and Android using Riverpod. A stateless **Express 5 / TypeScript** API of 28 endpoints on **Google Cloud Run**, scaling from zero. **Supabase PostgreSQL** across eight tables, plus **OpenAI**, **ElevenLabs** and **Razorpay**.

A voice turn is three timed stages: **Whisper** transcribes, **GPT-4o** generates under a language-specific persona prompt with the previous six turns supplied as context, and **ElevenLabs** synthesises the reply as speech.

Supporting systems: Google OAuth exchanged for a signed token; Razorpay subscriptions reconciled by HMAC-SHA256-verified webhooks, with failed verifications deliberately recorded as evidence that checking happens; five runtime feature flags; and a dependency-free administrative console with live metrics, content management, a webhook audit and an audit trail.

A deliberate clarification. There is no retrieval-augmented generation, no embedding and no vector index. Context is the previous turns replayed into the model's message array. Scripture citations come from the model's parameters, not from a retrieved corpus, and their accuracy was not formally verified.

5. Results

All 20 functional requirements are implemented; one (the audio player's previous/next controls) is partially met.

141 automated tests pass — 66 backend across 10 Jest suites, 75 client across 9 Flutter files, none failing, none skipped. Both suites were re-run for the report and the output captured as evidence.

The full subscription and conversation flow was verified end to end on physical hardware, including a real UPI payment and the resulting webhook reconciliation.

The measured latency fails its requirement, and finding that is the project's most useful result. NFR1 specifies a five-second pipeline. The live instance measures:

Stage	p50	Share
Speech to text (Whisper)	4,004 ms	15 %
AI response (GPT-4o)	3,471 ms	13 %
Text to speech (ElevenLabs)	19,618 ms	71 %
End to end	27,457 ms	

Sample: two voice turns — enough to establish an order of magnitude, not a distribution.

The three-to-six second figure quoted in the Phase 3 document and in the presentation deck is traceable to the administrative console's *demonstration fixtures* (3,030 ms), which are hand-written constants, not a measurement. The report records this correction in full.

The distribution is the useful part: latency is a speech-synthesis problem, not a model problem. And because the persona prompt specifies a two-to-four sentence reply while the model actually returns three paragraphs, the most probable single cause is the prompt rather than the vendor — the system is asking a speech engine to render four to five times more text than designed.

6. Honest limitations

Recorded in full in the report; the material ones are: latency fails NFR1 by roughly a factor of five; row-level security is enabled but unpolicied and bypassed by the API's service key, so authorization rests on application code with no test asserting isolation between users; a long-lived token and two analytics keys are committed to source control and must be rotated; there is no rate limiting and CORS is open; conversation transcripts are personal data stored indefinitely with no retention or deletion policy; test coverage stops at the service layer with no widget, integration or end-to-end tests; and three bundled hymns are CC BY-SA, which requires a credits screen the application does not yet have.

7. Demonstration

Recording	Duration	Link
Application	3 min 50 s	https://drive.google.com/file/d/124faeA_zm2HM7rPP_rWwJPsXSj6iN4QS/view
Administrative console	1 min 13 s	https://drive.google.com/file/d/1UeTyKBB8uDYdNhWjS14s2lbwYxO7xW6p/view

The console recording shows demonstration mode for its first fifty seconds and the live instance thereafter — the two halves report figures differing by an order of magnitude, for the reason given in Section 5.